

国科大作业考试资料《人工智能原理与算法》2024新编-第二次作业整理



AI Agent设计与评估

声明：本专题几乎每次作业都有两份答案作为参考，

后附两次考试原题+备期末考指南。

仅做参考交流使用。

1、对于下列活动，分别给出任务环境的PEAS描述，并按2.3.2节列出的性质进行分析：

- 足球运动
- 探索Titan的地下海洋
- 在互联网上购买AI旧书
- 打一场网球比赛
- 对着墙壁练网球
- 完成一次跳高
- 织一件毛衣
- 在一次拍卖中对一个物品投标

2、请回答下列问题，讨论Agent函数和Agent程序的区别：

- 是否有不止一个Agent程序可以实现给定的Agent函数？请举例说明，或者说明为什么不可能
- 有没有无法用任何Agent程序实现的Agent函数？
- 给定一个机器体系结构，能使每个Agent程序刚好实现一个Agent函数吗？
- 给定存储量为n比特的体系结构，可以有多少种可能的不同Agent程序？
- 假设我们让Agent程序固定不变，但机器速度提高，这会改变Agent函数吗？

3、为图2.2中描述的吸尘器世界实现一个可以进行性能度量的环境模拟器。你的实现必须是模块化的，以使传感器、执行器和环境特征（大小、形状、灰尘放置等）便于修改。

4、实现上述题目真空吸尘器环境中的简单反射Agent。用所有可能的初始灰尘分布和Agent位置，运行环境模拟器和

Agent。记录Agent在每种情况下的性能评分和总体平均评分。

参考一：

1.足球运动：

任务环境的PEAS描述：

性能度量：进球数、比赛胜率、球队排名等。

环境：足球场地、球员、球、裁判等。

执行器：运动员、教练等。

传感器：视觉感知（观察球场、队友和对手的位置）、听觉感知（听到队友的呼喊或教练的指示）等。

部分可观察，多Agent、随机的、延续式、动态的、连续的、未知的

探索Titan的地下海洋：

任务环境的PEAS描述：

性能度量：发现新的地下海洋特征、收集样本等。

环境：Titan的地下海洋、探测器、传感器等。

执行器：探测器的移动、样本采集、数据分析等。

传感器：探测器的摄像头、温度传感器、压力传感器等。

部分可观察、单Agent、随机的、延续式、动态的、连续的、未知的

在互联网上购买AI旧书：

任务环境的PEAS描述：

性能度量：购买成功率、购买价格、交易安全等。

环境：互联网购物平台、卖家、商品信息等。

执行器：购买者、互联网购物平台等。

传感器：互联网购物平台的界面、商品信息、用户评价等。

完全可观察、单Agent、

确定的、延续的、连续的、未知的

打一场网球比赛：

任务环境的PEAS描述：

性能度量：比赛得分、比赛胜率等。

环境：网球场、对手、球、裁判等。

执行器：选手、教练等。

传感器：视觉感知（观察球场、对手和球的位置）、听觉感知（听到球的声音）等。

部分可观察、多Agent、随机的、延续式、动态的、连续的、未知的

对着墙壁练网球：

任务环境的PEAS描述：

性能度量：击球准确度、击球力度等。

环境：墙壁、球、网球拍等。

执行器：运动员、教练等。

传感器：视觉感知（观察球的位置）、触觉感知（感受球的反弹力度）等。

完全可观察、单Agent、随机的、延续式、动态的、连续的、未知的

完成一次跳高：

任务环境的PEAS描述：

性能度量：跳高高度、技术动作完成度等。

环境：跳高场地、跳高杆、裁判等。

执行器：运动员、教练等。

传感器：视觉感知（观察杆的高度和位置）、听觉感知（听到教练的指导）等。

完全可观察、单Agent、随机的、延续式、动态的、连续的、未知的

织一件毛衣：

任务环境的PEAS描述：

性能度量：完成度、质量等。

环境：织毛衣的工具、毛线、图案等。

执行器：织毛衣的人、织毛衣的机器等。

传感器：视觉感知（观察编织的进度和质量）、触觉感知（感受毛线的质地和张力）等。

完全可观察、单Agent、随机的、延续式、静态的、连续的、未知的

在一次拍卖中对一个物品投标：

任务环境的PEAS描述：

性能度量：成功竞标率、竞标价格等。

环境：拍卖场地、竞争对手、拍卖物品等。

执行器：竞标者、拍卖师等。

传感器：视觉感知（观察竞争对手的动作和表情）、听觉感知（听到竞价声音）等。

完全可观察、多Agent、片段式、动态的、离散的、未知的

2。 - 是的，因为在实现Agent函数时，可以使用不同的算法、策略或方法来达到相同的目标。

- 是的，因为函数的要求超出了机器的能力或限制，或者是因为函数涉及到不可计算的问题。

例如，某些函数可能需要无限的计算资源或无法在有限的时间内得到解决，这样就无法通过任何Agent程序来实现。

- 不一定。机器的存储、处理能力以及其他硬件和软件限制都会对Agent程序的实现产生影响。

- 给定存储量为n比特的体系结构，可以有 2^n 种可能的不同Agent程序。这是因为每个比特可以有两种状态（0或1），而Agent程序可以由n个比特组成，因此总共有 2^n 种可能的组合。

- 不会。Agent函数是由Agent程序定义的，而机器速度的提高只会影响Agent程序的执行效率，而不会改变Agent函数本身。Agent函数描述了Agent在给定输入和环境条件下应该采取的行动，而机器速度的提高只会使Agent程序更快地执行这些行动，但不会改变Agent函数的定义或行为。

3.1. 创建一个环境类，该类包含吸尘器世界的特征，如大小、形状和灰尘放置等。在这种情况下，环境只有两个方格，可以用一个布尔数组来表示每个方格的状态（有灰尘或无灰尘）。

3.2. 在环境类中，实现传感器和执行器的功能。传感器可以读取吸尘器当前位置的信息，执行器可以根据传感器的信息执行吸尘器的动作。在这种情况下，传感器可以返回当前方格的状态（有灰尘或无灰尘），执行器可以执行两个动作：清扫和移动。

3.3 实现一个性能度量函数，用于评估Agent的性能。该函数可以根据吸尘器的动作和环境的状态来计算性能评分。在这种情况下，可以定义性能评分为吸尘器清扫的方格数目。

3.4 确保环境模拟器的实现是模块化的，使得传感器、执行器和环境特征可以方便地进行修改。这样可以轻松地改变方格的数量、形状和灰尘的放置，以及传感器和执行器的功能。

4.1. 创建一个名为"SimpleReflexAgent"的Agent类，该类包含以下属性和方法：

- 属性：
 - `position``: 代表Agent当前位置的变量，可以是0或1，分别对应两个方格。
- 方法：
 - `choose_action(percept)``: 根据传感器的信息选择合适的动作。

在这种情况下，可以简单地根据当前方格的状态（有灰尘或无灰尘）来选择动作。如果当前方格有灰尘，则返回"clean"（清扫）动作；如果当前方格无灰尘，则返回"move"（移动）动作。

4.2. 在每种情况下，使用环境模拟器和Agent来模拟真空吸尘器世界，并记录Agent在每种情况下的性能评分和总体平均评分：

首先，生成所有可能的初始灰尘分布和Agent位置的组合。

对于每种组合，创建一个"VacuumWorld"环境实例和一

个"SimpleReflexAgent" Agent实例。

在每个时间步中，先调用环境实例的`get\_percept()`方法获取传感器的信息，然后调用Agent实例的`choose\_action()`方法选择动作，最后调用环境实例的`perform\_action()`方法执行动作。

在每个时间步结束后，检查环境实例的性能评分，并记录下来。

重复上述步骤，直到所有组合都被模拟完毕。

计算所有性能评分的总和，并除以模拟的组合数，得到总体平均评分。

```
class Environment:
    def __init__(self, size, dust_distribution):
        self.size = size
        self.dust_distribution = dust_distribution
        self.agent_position = (0, 0)
        self.score = 0

    def place_dust(self):
        # 在环境中放置灰尘

    def update_score(self):
        # 更新性能评分

    def move_agent(self, action):
        # 移动Agent的位置

    def clean_dust(self):
        # 清理灰尘

    def simulate(self):
        # 模拟真空吸尘器环境和Agent的行为
        for each possible initial dust distribution and agent position:
            self.place_dust()
            while True:
                if self.agent_position has dust:
                    self.clean_dust()
                else:
                    # 根据Agent的位置和环境状态作出决策
                    action = self.agent.decide_action()
                    self.move_agent(action)
                self.update_score()
                if self.is_environment_clean():
                    break
            record the score

class Agent:
    def __init__(self):
        pass

    def decide_action(self):
        # 根据环境状态作出决策，例如向右移动、向左移动、清理灰尘等

def main():
    environment = Environment(size, dust_distribution)
    environment.simulate()
    calculate average score

main()
```

参考二：

Agent	Performance	Environment	Actuators	Sensors	性质分析
足球运动	进球得分、 控球次数、 续航时间、 运球速度	场地草坪、 足球材质、 队友配合	机械腿、 马达	红外传感器、 相机、GPS、 速度传感器	部分可观察、 多Agent、随机、 动态的、连续的、 未知的
探索Titan 的地下海洋	下沉深度、 行走速度、 续航时间、 地形探测能力	海水、生物、 压强、 光照程度	发动机、 机械臂、 灯泡	压强传感器、 光照传感器、 温度传感器、 摄像头、GPS、 红外传感器	部分可观察、 单Agent、随机、 动态的、连续的、 未知的
在互联网上 购买电子书	商品质量、 商品价格、 搜索精良、 用户满意度	用户评价、 旧书总量、 互联网平台	"购买"及"付款" 按钮	语言处理、 技术和图像处理	部分可观察、 单Agent、确定、 静态的、离散、 未知的

Agent	Performance	Environment	Actuators	Sensors	性质分析
打一场网球 比赛	得分、击球力度、 击球角度精确度、 续航时间、 接球精度	球拍、网球、 与中网距离、 对手击球角度 和力度	机械臂、 马达	摄像头、 压力传感器、 速度传感器	完全可观察、 多Agent、随机、 片段式的、初始、 连续的、未知的
对着墙壁 练网球	击球力度、 击球角度精确度、 续航时间、 接球精度	墙壁的硬度、 球拍的质量、 网球的质量	机械臂 与达	摄像头、 压力传感器、 速度传感器	完全可观察、 单Agent、随机、 片段式的、动态、 连续的、未知的
完成一次 跳高	所跳高度、 所跳高度精度	地面温度、 柔软情况	机械臂 马达	"肌腱"电机、 "膝盖"电机、 的屈伸程度	完全可观察的、 单Agent、随机、 连续的、动态、 连续的、已知

Agent	Performance	Environment	Actuators	Sensors	性质分析
织一件 毛衣	毛衣工艺复杂程度、 毛衣与预期大小的 符合度、 织毛衣的速度	毛线的质量、 种类	机械臂、 电机	摄像头、 红外传感器、 压力传感器	完全可观察、 单Agent、确定、 连续的、初始、 连续的、已知
在一次拍 卖中为一个 物品投标	所拍物品的 价格、对物品 价格的精度、	对手报价、 竞拍的商品	写入投标价格、 录入竞标系统	图像处理、 语音识别、 语音理解	完全可观察的、 多Agent、随机、 连续的、静态、 离散的、未知

二.

- 是否有不止一个Agent程序可以实现给定的Agent函数？
请举例说明，或者说明为什么不可能

是例如添加一些代理程序，只输入不会影响输出的空值。

- 有没有无法用任何Agent程序实现的Agent函数？

有，例如图灵机

- 给定一个机器体系结构，能使每个Agent程序刚好实现一个Agent函数吗？

能，Agent的行为可被结构和程序固定。

- 给定存储量为n比特的体系结构，可以有多少种可能的不同Agent程序？

有 2^n 种不同的程序，但大部分都无法执行，因为每个程序占用n个内存导致内存

- 假设我们让Agent程序固定不变，但机器速度提高，这会改变Agent函数吗？

这取决于程序和环境。若环境是动态的，机器速度提高意味着动作改变，即Agent函数改变。
若环境是静态的，且程序不在意运行时间，则Agent函数保持不变。

三.

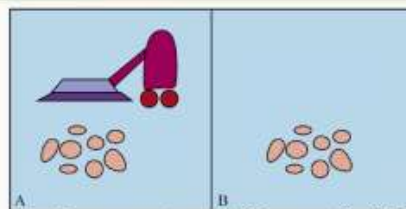


图2-2 一个只有两个方格的真空吸尘器世界

假设

- ① 在1000个时间同步的“生命周期”内，性能度量在每个时间步内为每个干净的方格奖励1分
- ② 环境的“地理信息”是先验的，但灰尘的分布和智能体的初始位置不是先验的。干净的方格会继续保持干净，吸尘(Suck)动作会清除当前方格。向左(Left)或向右(Right)的动作使智能体移动一个方格。如果该动作会让智能体移动到环境之外，智能体会保持在原来的位置。
- ③ 可用的动作仅有向右(Right)、向左(Left)和吸尘(Suck)
- ④ 智能体能够正确感知其位置及其该位置是否有灰尘

则该真空吸尘器世界的简单智能体函数的部分表项如下所示:

感知序列	动作
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Clean]	Suck

注:如果当前方格是脏的,智能体就会进行清理,否则将移动到另一个方格

将当前的感知作为传感器的输入,并将动作返回给执行器,
环境模拟器如下:

```
function Table-Driven-Agent (percept) returns 一个动作
    persistent: percepts, 初始为空的序列
               table, 以感知序列为索引的动作表 初始完全确定
    将 percept 添加到 percepts 的末尾
    actions  $\rightarrow$  Lookup(percepts, table)
    return action
```

CSDN @MatsumotoChr

四.

简单反射型智能体的代码如下:

```
function Reflex-Vacuum-Agent [location, status] returns 一个动作
    int scale = 0;
    for (int n = 1; n <= 1000; n++) {
        if status = Dirty then
            scale + = 1  若吸尘, 则奖励1分
            return Suck
        else if location = A then return Right 右移
        else if location = B then return Left 左移
    }
    scale_avg = scale / n  计算总体平均得分
```

CSDN @MatsumotoChr